

Microsoft Fabric 전문가가 번역하고 설명하는 핸드온 가이드입니다.

핸즈온 랩: T-SQL을 사용하여 웨어하우스로 데이터 로드하기

Microsoft Fabric에서 **data warehouse**는 대규모 분석을 위한 관계형 데이터베이스를 제공합니다. Lakehouse에 정의된 테이블에 대한 기본 읽기 전용 SQL 엔드포인트와 달리, Data Warehouse는 테이블의 데이터를 삽입(INSERT), 업데이트(UPDATE), 삭제(DELETE)하는 능력을 포함하여 완전한 SQL 의미 체계(semantics)를 제공합니다.

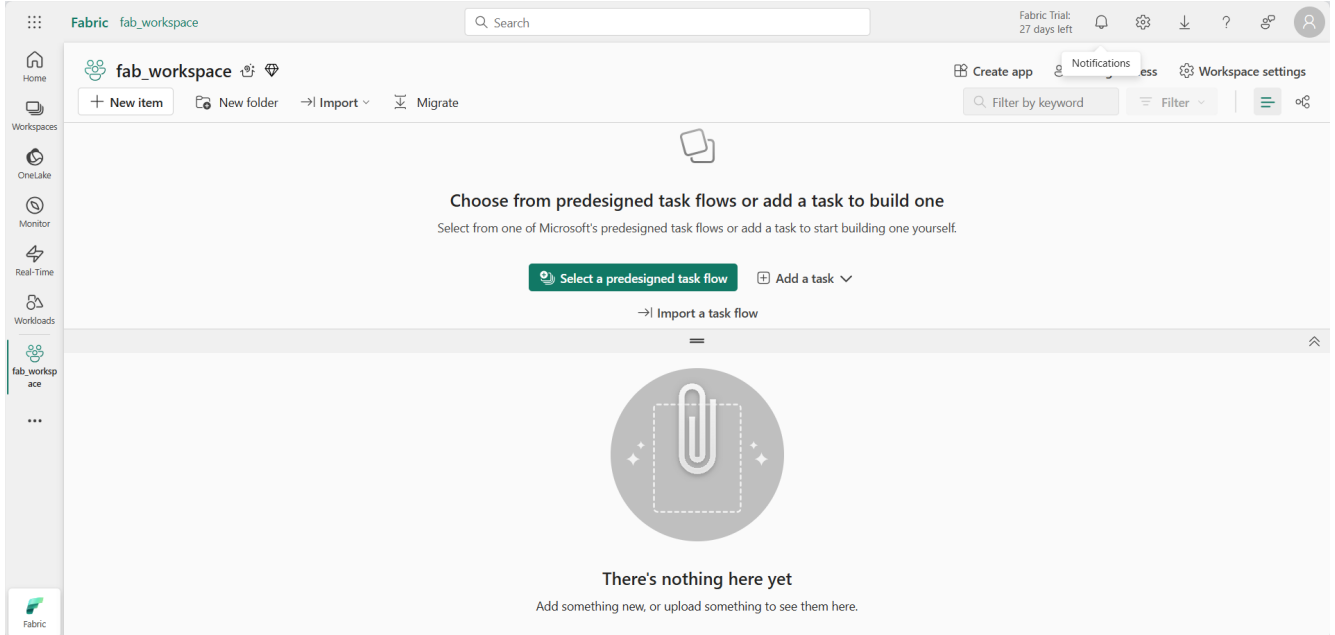
이 실습을 완료하는 데 약 **30분**이 소요됩니다.

참고: 이 실습을 완료하려면 [Microsoft Fabric 평가판](#)이 필요합니다.

Workspace 만들기

Fabric에서 데이터 작업을 시작하기 전에 Fabric 평가판이 활성화된 Workspace를 만들어야 합니다.

1. 웹 브라우저에서 [Microsoft Fabric 홈페이지](https://app.fabric.microsoft.com/home?experience=fabric) `https://app.fabric.microsoft.com/home?experience=fabric` 로 이동하여 Fabric 자격 증명으로 로그인합니다.
2. 왼쪽 메뉴 모음에서 **Workspaces** 아이콘()을 선택합니다.
3. 원하는 이름으로 새 Workspace를 만듭니다. **Advanced** 섹션에서 Fabric 용량(*Trial*, *Premium*, 또는 *Fabric*)을 포함하는 라이선스 모드를 선택해야 합니다.
4. 새 Workspace가 열리면 처음에는 비어 있어야 합니다.

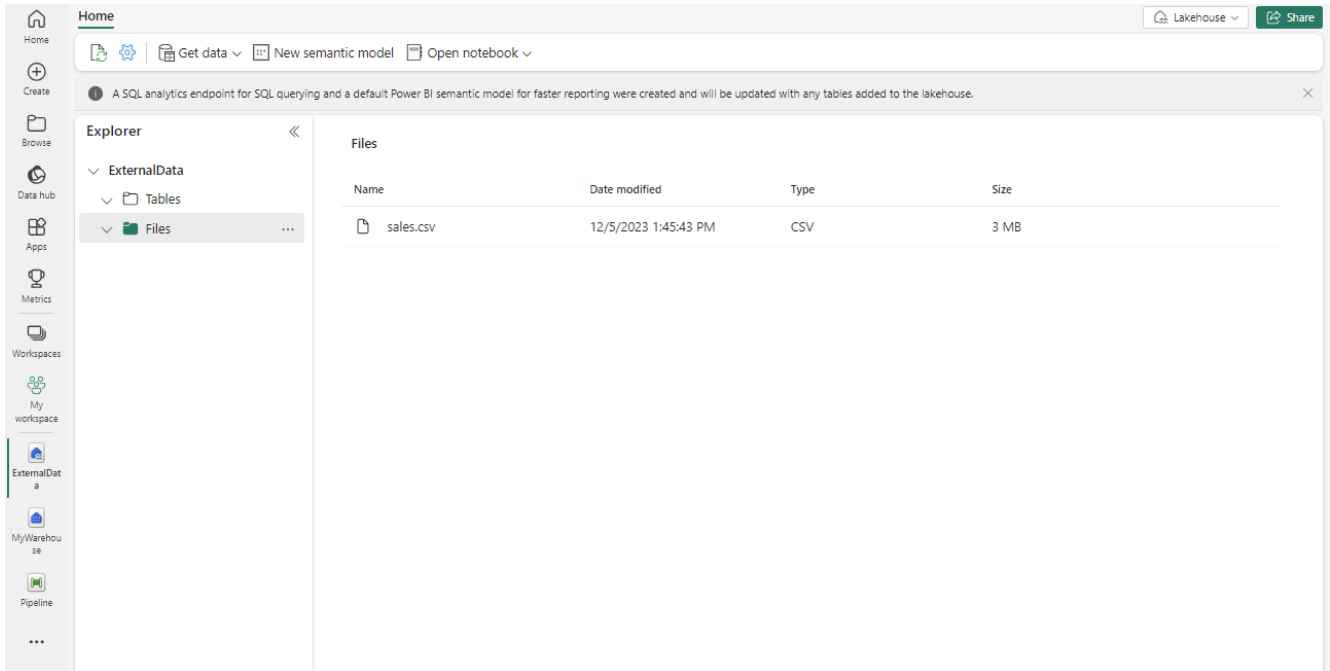


Lakehouse 만들기 및 파일 업로드

개념 설명: 이 시나리오에서는 웨어하우스를 로드하는 데 사용할 원본 데이터가 없습니다. 따라서 먼저 데이터를 수집할 장소가 필요합니다. 여기서는 **Lakehouse**를 원시 데이터 파일을 위한 **랜딩(Landing)** 또는 **스테이징(Staging)** 영역으로 사용할 것입니다. 이는 데이터를 최종 목적지(Warehouse)로 로드하기 전에 임시로 저장하고 준비하는 일반적인 데이터 엔지니어링 패턴입니다.

1. **+ New item**을 선택하고 원하는 이름으로 새 **Lakehouse**를 만듭니다.

- 이 실습을 위한 파일을 <https://github.com/MicrosoftLearning/dp-data/raw/main/sales.csv> 에서 다운로드합니다.
- Lakehouse가 열려 있는 웹 브라우저 탭으로 돌아가, **Explorer** 창의 **Files** 폴더에 있는 ... 메뉴에서 **Upload** 및 **Upload files**를 선택한 다음, 로컬 컴퓨터에서 **sales.csv** 파일을 Lakehouse에 업로드합니다.
- 파일이 업로드된 후, **Files**를 선택하여 CSV 파일이 업로드되었는지 확인합니다.



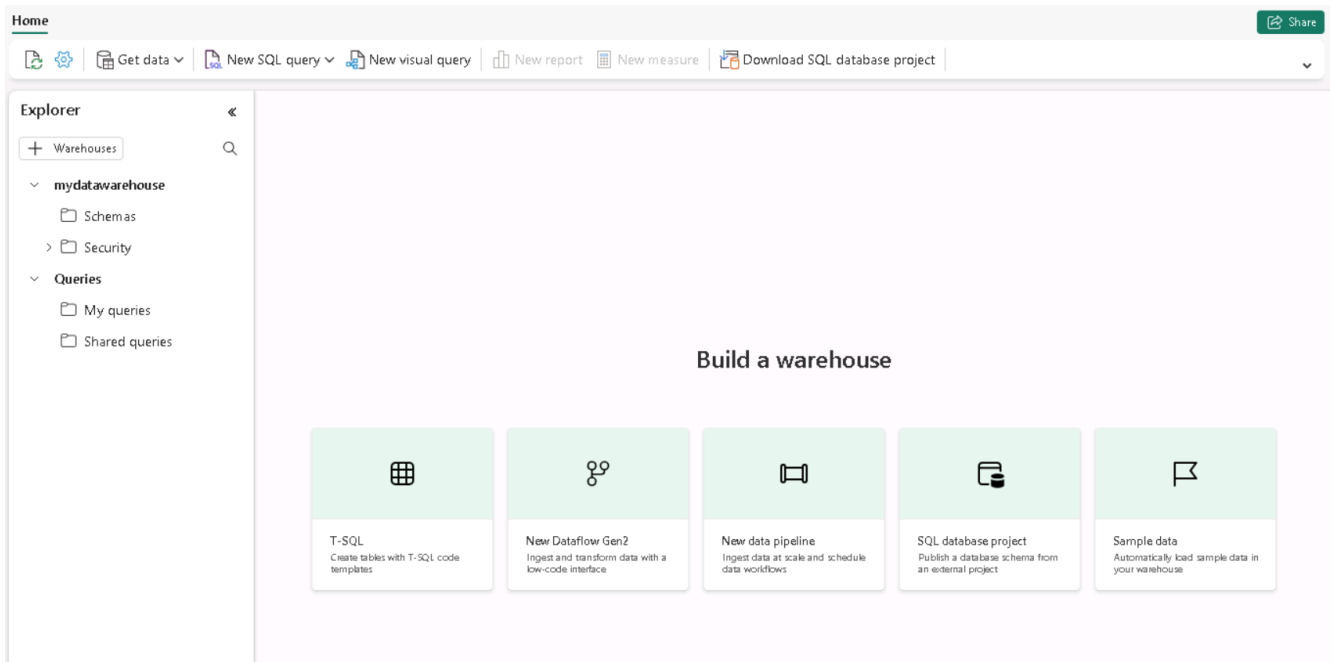
Lakehouse에 테이블 만들기

- Explorer** 창의 **sales.csv** 파일에 대한 ... 메뉴에서 **Load to tables**를 선택한 다음, **New table**을 선택합니다.
- Load file to new table** 대화 상자에서 다음 정보를 제공합니다.
 - New table name:** `staging_sales`
 - Use header for columns names:** 선택됨
 - Separator:** `,`
- Load**를 선택합니다. 이제 원시 데이터가 `staging_sales` 라는 스테이징 테이블에 로드되었습니다.

Warehouse 만들기

이제 Workspace, Lakehouse, 그리고 필요한 데이터가 담긴 sales 테이블이 준비되었으니, 데이터 웨어하우스를 만들 차례입니다.

- 왼쪽 메뉴 모음에서 **Create**를 선택합니다. *New* 페이지의 *Data Warehouse* 섹션에서 **Warehouse**를 선택하고, 원하는 고유한 이름을 지정합니다.
- 약 1분 후, 새로운 빈 웨어하우스가 생성됩니다.



Fact 테이블, Dimension 테이블 및 View 만들기

이제 판매 데이터를 위한 Fact 테이블과 Dimension 테이블을 만들겠습니다. 또한 Lakehouse를 가리키는 View도 만들 것입니다. 이렇게 하면 나중에 데이터를 로드하는 데 사용할 저장 프로시저(Stored Procedure)의 코드가 단순화됩니다.

1. Workspace에서 방금 만든 웨어하우스를 선택합니다.
2. 웨어하우스 톨바에서 **New SQL query**를 선택한 다음, 다음 쿼리를 복사하여 실행합니다.

```
CREATE SCHEMA [Sales]
GO

IF NOT EXISTS (SELECT * FROM sys.tables WHERE name='Fact_Sales' AND
SCHEMA_NAME(schema_id)='Sales')
CREATE TABLE Sales.Fact_Sales (
    CustomerID VARCHAR(255) NOT NULL,
    ItemID VARCHAR(255) NOT NULL,
    SalesOrderNumber VARCHAR(30),
    SalesOrderLineNumber INT,
    OrderDate DATE,
    Quantity INT,
    TaxAmount FLOAT,
    UnitPrice FLOAT
);

IF NOT EXISTS (SELECT * FROM sys.tables WHERE name='Dim_Customer' AND
SCHEMA_NAME(schema_id)='Sales')
CREATE TABLE Sales.Dim_Customer (
    CustomerID VARCHAR(255) NOT NULL,
    CustomerName VARCHAR(255) NOT NULL,
    EmailAddress VARCHAR(255) NOT NULL
);
```

```

ALTER TABLE Sales.Dim_Customer add CONSTRAINT PK_Dim_Customer PRIMARY KEY
NONCLUSTERED (CustomerID) NOT ENFORCED
GO

IF NOT EXISTS (SELECT * FROM sys.tables WHERE name='Dim_Item' AND
SCHEMA_NAME(schema_id)='Sales')
    CREATE TABLE Sales.Dim_Item (
        ItemID VARCHAR(255) NOT NULL,
        ItemName VARCHAR(255) NOT NULL
    );

ALTER TABLE Sales.Dim_Item add CONSTRAINT PK_Dim_Item PRIMARY KEY NONCLUSTERED
(ItemID) NOT ENFORCED
GO

```

코드 설명:

- `CREATE SCHEMA [Sales]`: Sales라는 이름의 새 스키마를 만들어 관련 테이블들을 논리적으로 그룹화합니다.
- `IF NOT EXISTS (...) CREATE TABLE ...`: 테이블이 아직 존재하지 않을 경우에만 생성하도록 하여, 스크립트를 여러 번 실행해도 오류가 발생하지 않도록 합니다.
- `Sales.Fact_Sales`: 숫자 측정값(수량, 단가 등)을 담는 사실(Fact) 테이블을 생성합니다.
- `Sales.Dim_Customer`, `Sales.Dim_Item`: 분석의 기준이 되는 속성 정보(고객 이름, 아이템 이름 등)를 담는 차원(Dimension) 테이블들을 생성합니다.
- `ALTER TABLE ... ADD CONSTRAINT ... NOT ENFORCED`: 기본 키(Primary Key) 제약 조건을 추가하지만, `NOT ENFORCED` 옵션을 사용하여 데이터 로드 시 제약 조건 검사를 강제하지 않습니다. 이는 데이터 웨어하우스 환경에서 대량 데이터 로드 성능을 향상시키기 위해 흔히 사용되는 기법입니다.

3. **Explorer**에서 **Schemas >> Sales >> Tables**로 이동하여 방금 만든 `Fact_Sales`, `Dim_Customer`, `Dim_Item` 테이블을 확인합니다.

4. 새로운 **New SQL query** 편집기를 열고 다음 쿼리를 복사하여 실행합니다. `<your lakehouse name>` 부분을 직접 만든 Lakehouse의 이름으로 업데이트해야 합니다.

개념 설명: 교차 데이터베이스 쿼리 (Cross-database query)

이 단계는 Microsoft Fabric의 핵심적인 강력함을 보여줍니다. **Data Warehouse**에서 `CREATE VIEW` 문을 실행하면, 물리적으로 다른 객체인 **Lakehouse**에 있는 테이블(`[<your lakehouse name>].[dbo].[staging_sales]`)을 직접 참조하고 있습니다. 이는 Fabric의 통합된 OneLake 스토리지를 기반으로 하기 때문에 가능하며, 데이터를 복제할 필요 없이 웨어하우스에서 레이크하우스의 데이터에 즉시 접근할 수 있게 해주는 매우 효율적인 기능입니다.

```

CREATE VIEW Sales.Staging_Sales
AS
SELECT * FROM [<your lakehouse name>].[dbo].[staging_sales];

```

5. **Explorer**에서 **Schemas >> Sales >> Views**로 이동하여 방금 만든 `Staging_Sales` 뷰를 확인합니다.

웨어하우스로 데이터 로드하기

개념 설명: 저장 프로시저 (Stored Procedure)

이제 웨어하우스로 데이터를 로드하는 로직을 **저장 프로시저(Stored Procedure)**로 만들 것입니다. 저장 프로시저를 사용하면 데이터 로드와 관련된 복잡한 T-SQL 로직을 하나의 재사용 가능한 객체로 캡슐화할 수 있습니다. 또한, `@OrderYear` 와 같은 매개변수를 사용하여 특정 연도의 데이터만 로드하는 등 유연한 실행이 가능해집니다.

1. 새 **New SQL query** 편집기를 만들고 다음 쿼리를 복사하여 실행합니다.

```
CREATE OR ALTER PROCEDURE Sales.LoadDataFromStaging (@OrderYear INT)
AS
BEGIN
    -- Load data into the Customer dimension table
    INSERT INTO Sales.Dim_Customer (CustomerID, CustomerName, EmailAddress)
    SELECT DISTINCT CustomerName, CustomerName, EmailAddress
    FROM [Sales].[Staging_Sales]
    WHERE YEAR(OrderDate) = @OrderYear
    AND NOT EXISTS (
        SELECT 1
        FROM Sales.Dim_Customer
        WHERE Sales.Dim_Customer.CustomerName = Sales.Staging_Sales.CustomerName
        AND Sales.Dim_Customer.EmailAddress = Sales.Staging_Sales.EmailAddress
    );

    -- Load data into the Item dimension table
    INSERT INTO Sales.Dim_Item (ItemID, ItemName)
    SELECT DISTINCT Item, Item
    FROM [Sales].[Staging_Sales]
    WHERE YEAR(OrderDate) = @OrderYear
    AND NOT EXISTS (
        SELECT 1
        FROM Sales.Dim_Item
        WHERE Sales.Dim_Item.ItemName = Sales.Staging_Sales.Item
    );

    -- Load data into the Sales fact table
    INSERT INTO Sales.Fact_Sales (CustomerID, ItemID, SalesOrderNumber,
    SalesOrderLineNumber, OrderDate, Quantity, TaxAmount, UnitPrice)
    SELECT CustomerName, Item, SalesOrderNumber, CAST(SalesOrderLineNumber AS INT),
    CAST(OrderDate AS DATE), CAST(Quantity AS INT), CAST(TaxAmount AS FLOAT),
    CAST(UnitPrice AS FLOAT)
    FROM [Sales].[Staging_Sales]
    WHERE YEAR(OrderDate) = @OrderYear;
END
```

코드 설명:

- `CREATE OR ALTER PROCEDURE ... (@OrderYear INT): LoadDataFromStaging`이라는 이름의 저장 프로시저를 생성하거나, 이미 존재하면 수정합니다. 이 프로시저는 정수형 매개변수 `@OrderYear`를 입력받습니다.
- `INSERT INTO ... SELECT DISTINCT ...`: 스테이징 테이블에서 고유한(DISTINCT) 고객 및 아이템 정보를 가져와 각각의 차원 테이블로 로드합니다.

- `AND NOT EXISTS (...)`: 이 구문은 차원 테이블에 이미 동일한 고객이나 아이템이 존재하는지 확인하여, 중복된 데이터가 삽입되는 것을 방지합니다.
- `INSERT INTO Sales.Fact_Sales ...`: 마지막으로, 해당 연도(`@OrderYear`)의 모든 판매 데이터를 스테이징 테이블에서 가져와 `Fact_Sales` 테이블로 로드합니다. 데이터 타입을 명시적으로 변환(`CAST`)하여 데이터 정합성을 보장합니다.

2. 새 **New SQL query** 편집기를 만들고 다음 쿼리를 복사하여 실행합니다. 이 코드는 위에서 만든 저장 프로시저를 호출하여 2021년도 데이터를 로드합니다.

```
EXEC Sales.LoadDataFromStaging 2021
```

분석 쿼리 실행하기

이제 웨어하우스의 데이터가 올바른지 검증하기 위해 몇 가지 분석 쿼리를 실행해 봅시다.

1. 새 SQL 쿼리에서 다음 쿼리를 복사하여 실행합니다.

```
SELECT c.CustomerName, SUM(s.UnitPrice * s.Quantity) AS TotalSales
FROM Sales.Fact_Sales s
JOIN Sales.Dim_Customer c
ON s.CustomerID = c.CustomerID
WHERE YEAR(s.OrderDate) = 2021
GROUP BY c.CustomerName
ORDER BY TotalSales DESC;
```

결과 분석: 이 쿼리는 2021년의 총 판매액 기준 상위 고객을 보여줍니다. 결과에서 **Jordan Turner**가 총 판매액 **14686.69**로 가장 높은 고객임을 확인할 수 있습니다.

2. 새 SQL 쿼리에서 다음 쿼리를 복사하여 실행합니다.

```
SELECT i.ItemName, SUM(s.UnitPrice * s.Quantity) AS TotalSales
FROM Sales.Fact_Sales s
JOIN Sales.Dim_Item i
ON s.ItemID = i.ItemID
WHERE YEAR(s.OrderDate) = 2021
GROUP BY i.ItemName
ORDER BY TotalSales DESC;
```

~~~ **\*\*결과 분석:\*\*** 이 쿼리는 2021년의 총 판매액 기준 상위 판매 아이템을 보여줍니다. 결과는 **\*Mountain-200 bike\*** 모델(블랙 및 실버 색상)이 2021년에 가장 인기 있는 아이템이었음을 시사합니다.

3. 새 SQL 쿼리에서 다음 쿼리를 복사하여 실행합니다.

```
WITH CategorizedSales AS (
SELECT
CASE
WHEN i.ItemName LIKE '%Helmet%' THEN 'Helmet'
WHEN i.ItemName LIKE '%Bike%' THEN 'Bike'
WHEN i.ItemName LIKE '%Gloves%' THEN 'Gloves'
ELSE 'Other'
```

```

        END AS Category,
        c.CustomerName,
        s.UnitPrice * s.Quantity AS Sales
FROM Sales.Fact_Sales s
JOIN Sales.Dim_Customer c
ON s.CustomerID = c.CustomerID
JOIN Sales.Dim_Item i
ON s.ItemID = i.ItemID
WHERE YEAR(s.OrderDate) = 2021
),
RankedSales AS (
    SELECT
        Category,
        CustomerName,
        SUM(Sales) AS TotalSales,
        ROW_NUMBER() OVER (PARTITION BY Category ORDER BY SUM(Sales) DESC) AS
SalesRank
    FROM CategorizedSales
    WHERE Category IN ('Helmet', 'Bike', 'Gloves')
    GROUP BY Category, CustomerName
)
SELECT Category, CustomerName, TotalSales
FROM RankedSales
WHERE SalesRank = 1
ORDER BY TotalSales DESC;

```

### 코드 설명 (Common Table Expression - CTE):

- `WITH CategorizedSales AS (...)`: 첫 번째 CTE는 `ItemName` 에 'Helmet', 'Bike' 등이 포함되어 있는지에 따라 동적으로 `Category` 열을 생성합니다.
- `RankedSales AS (...)`: 두 번째 CTE는 첫 번째 CTE의 결과를 사용하여 각 `Category` 내에서 고객별 총 판매액 순위(`SalesRank`)를 매깁니다. `ROW_NUMBER() OVER (PARTITION BY Category ...)` 는 카테고리별로 파티션을 나누고 그 안에서 판매액이 높은 순으로 1, 2, 3... 순위를 부여하는 윈도우 함수입니다.
- `SELECT ... FROM RankedSales WHERE SalesRank = 1`: 최종적으로, 각 카테고리에서 판매 순위가 1위인 고객만 선택하여 보여줍니다.

**결과 분석:** 이 쿼리 결과는 각 카테고리(Bike, Helmet, Gloves)별 상위 고객을 보여줍니다. 예를 들어, **Joan Coleman**은 **Gloves** 카테고리의 상위 고객입니다.

## 리소스 정리

이 실습에서는 Lakehouse와 여러 테이블을 포함하는 Data Warehouse를 만들었습니다. 데이터를 수집하고, 교차 데이터베이스 쿼리를 사용하여 Lakehouse에서 Warehouse로 데이터를 로드했습니다. 또한 쿼리 도구를 사용하여 분석 쿼리를 수행했습니다.

데이터 웨어하우스 탐색을 마쳤다면 이 실습을 위해 만든 Workspace를 삭제할 수 있습니다.

1. 왼쪽 막대에서 Workspace 아이콘을 선택하여 포함된 모든 항목을 봅니다.
2. **Workspace settings**를 선택하고 **General** 섹션에서 아래로 스크롤하여 **Remove this workspace**를 선택합니다.
3. **Delete**를 선택하여 Workspace를 삭제합니다.